PROJET FINAL

Déployer une application Full-Stack sur AWS

🚫 **N'utilisez PAS Elastic Beanstalk.**

Elastic Beanstalk masque l'infrastructure. L'objectif de ce projet est précisément que vous construisiez et contrôliez chaque composant vous-même — VPC, ALB, ASG, Launch Template. Si vous utilisez Beanstalk, rien de tout cela ne pourra être évalué.

1. De quoi s'agit-il ?

Dans ce projet, vous allez déployer une application web full-stack complète sur AWS depuis zéro, exactement comme le ferait un ingénieur cloud dans le monde professionnel.

Vous construirez toute l'infrastructure vous-même : le réseau, les serveurs, le répartiteur de charge, l'auto scaling et la base de données. Votre application doit être accessible depuis internet et rester opérationnelle même si un serveur tombe en panne.

💡 Imaginez que vous construisez l'infrastructure cloud d'une jeune startup. Tout doit être correctement connecté, sécurisé et résilient.

2. Ce que vous devez construire

Votre infrastructure doit comporter les composants suivants. Chacun d'entre eux a déjà été vu en cours.

① Le Réseau — VPC

Vous allez créer votre propre réseau isolé sur AWS. C'est le socle de tout le reste.

✏️ Nous avons vu cela dans le lab VPC, vous avez déjà créé un VPC, des sous-réseaux, une Internet Gateway et des tables de routage.

- 1 VPC avec le bloc CIDR 10.0.0.0/16
- 2 Zones de Disponibilité (AZ-A et AZ-B) pour la résilience

- Dans chaque AZ : 1 sous-réseau public + 1 sous-réseau privé (4 sous-réseaux au total)
- 1 Internet Gateway — pour laisser entrer le trafic internet dans le VPC
- 1 NAT Gateway (dans le sous-réseau public) — pour que les serveurs privés puissent accéder à internet
- Des tables de routage correctes pour les sous-réseaux publics et privés
- Des pare-feux (Security Groups) pour chaque couche — ALB, EC2 et RDS — pour contrôler qui peut communiquer avec qui

📌 Les sous-réseaux publics accueillent les ressources exposées à internet (ALB, NAT GW). Les sous-réseaux privés hébergent les ressources qui ne doivent jamais être directement accessibles (backend EC2, base RDS).

② Le Backend — EC2 + Load Balancer + Auto Scaling

Votre application backend (API) tournera sur des instances EC2 placées dans les sous-réseaux privés, derrière un répartiteur de charge.

✏ Nous avons vu cela dans les labs EC2, Load Balancer et Auto Scaling — vous avez déjà lancé des instances, configuré un ALB et mis en place un ASG.

- 1 Application Load Balancer (ALB) placé dans les deux sous-réseaux publics
- 1 Target Group avec un health check sur votre API (ex. : GET /health → 200 OK)
- 1 Launch Template qui installe et démarre votre application automatiquement au démarrage (User Data)
- 1 Auto Scaling Group (ASG) : min 2 — souhaité 2 — max 4, réparti sur les deux sous-réseaux privés
- 1 Scaling Policy basée sur l'utilisation CPU (monter en charge si CPU > 70 %)

📌 Les utilisateurs ne communiquent jamais directement avec vos instances EC2. Tout le trafic passe par l'ALB → Target Group → EC2. Si une instance devient défaillante, l'ASG la remplace automatiquement.

💡 Conseil de déploiement — Backend (script User Data)

Votre Launch Template doit inclure un script User Data qui s'exécute automatiquement au démarrage de chaque instance EC2. Ce script doit :

```
#!/bin/bash
sudo apt update -y && sudo apt install -y git nodejs npm   # (ou python3,
java, etc.)
cd /home/ubuntu
git clone https://github.com/<votre-nom>/<votre-repo>.git app
cd app
npm install
export DB_HOST=<endpoint-rds>
export DB_PASS=<mot-de-passe-db>
```

```
npm start &
```

Chaque nouvelle instance créée par l'ASG récupèrera automatiquement votre code et démarrera le serveur — sans connexion SSH manuelle.

③ Le Frontend

Votre frontend (HTML/CSS/JS) doit être servi par une instance EC2 dédiée placée dans le sous-réseau public, et doit être accessible depuis un navigateur.

Nous avons vu cela dans le lab EC2 — lancer une instance dans un sous-réseau public et y accéder en HTTP, c'est exactement ce que vous avez fait.

- 1 instance EC2 dans un sous-réseau public — sert vos pages HTML/CSS/JS
- Une règle de pare-feu (Security Group) autorisant le trafic HTTP entrant depuis internet
- Le frontend doit appeler le backend via le nom DNS de l'ALB — jamais directement via l'IP d'une instance EC2

L'instance EC2 frontend est dans le sous-réseau public pour que les utilisateurs puissent y accéder. Les instances EC2 backend restent dans le sous-réseau privé — les utilisateurs n'y accèdent jamais directement.

Conseil de déploiement — Frontend (SSH d'abord, puis script)

Approche recommandée — procédez en deux étapes :

1. Connectez-vous en SSH à l'instance EC2 frontend et exécutez vos commandes manuellement — installez un serveur web (nginx ou Apache), copiez vos fichiers, lancez le service. Vérifiez que tout fonctionne dans le navigateur.
2. Une fois que tout fonctionne, copiez ces commandes dans un script User Data de votre Launch Template. Votre serveur frontend sera ainsi reproductible et vous ne dépendrez plus d'une seule instance.

Commencer par SSH permet de déboguer de manière interactive. Une fois que vous êtes sûr que les étapes sont correctes, les transformer en script devient simple.

④ La Base de Données — Amazon RDS

Votre application doit stocker ses données dans une base de données relationnelle. Cette base ne doit jamais être directement accessible depuis internet.

Nous avons vu cela dans le lab RDS — vous avez déjà lancé une instance RDS, l'avez placée dans des sous-réseaux privés et vous y êtes connecté depuis une instance EC2.

- Amazon RDS — moteur MySQL ou PostgreSQL
- Type d'instance : db.t3.micro (éligible au Free Tier)

- Déployée dans un DB Subnet Group utilisant les deux sous-réseaux privés
- Le mot de passe de la base ne doit PAS être écrit en dur dans le code — utilisez une variable d'environnement

🔒 L'accès à RDS est contrôlé par un pare-feu (Security Group). Configurez-le pour n'accepter les connexions que depuis le Security Group des instances EC2 backend — pas depuis internet, pas depuis votre ordinateur, uniquement depuis vos serveurs backend.

🔒 Note d'évaluation — Les règles des Security Groups seront vérifiées

Le correcteur ouvrira chaque Security Group pendant la démonstration et vérifiera les règles. Assurez-vous que :

- Le Security Group de l'ALB autorise uniquement le trafic HTTP entrant (port 80) depuis internet (0.0.0.0/0)
- Le Security Group des EC2 backend n'autorise le trafic entrant que depuis le Security Group de l'ALB — pas depuis internet
- Le Security Group de RDS n'autorise le trafic entrant que depuis le Security Group des EC2 backend — pas depuis internet, pas depuis votre ordinateur
- Le Security Group de l'EC2 frontend autorise le trafic HTTP (port 80) depuis internet, et le SSH (port 22) uniquement si nécessaire pour le débogage

Une règle trop permissive comme 0.0.0.0/0 sur le port 3306 ou 5432 vous coûtera des points — même si l'application fonctionne.

Bonne chance ! ☁

Tout ce dont vous avez besoin se trouve déjà dans vos labs — reprenez-les, relisez-les, et appliquez ce que vous avez appris.